

Recent Increase in Software Supply Chain Threat Activity

Hi Andrew, thanks for copying me.

I think we should assume that we will eventually get hit by a supply chain attack, because that assumption forces us to act in a way that lowers the blast radius of a successful attack. It is not possible to anticipate which npm dependencies will get attacked next, but it is possible to harden our environment to reduce impact.

We have a few levers we can pull. They fall into two broad categories:

- **Proactive defenses:** safer-by-default controls that reduce the odds or impact of compromise.
- **Reactive defenses:** after-the-fact investigation and response capabilities.

I also want to mention that last week, the Shai-Hulud malware source code got leaked on Twitter, so anyone with a computer and basic knowledge of npm packages can start staging attacks. I was able to get a copy on my non-work Linux laptop before it was taken down, and I am betting that hackers are already adapting it to other languages and package managers.

Proactive Defenses

Sandboxed Development Environments

- Use a sandboxed development environment, such as GitHub Codespaces or a stripped-down VM that contains only what developers need.
- This is useful because, if or when we get hit, the impact should be smaller.
- For example, a developer with 10+ API keys on their local laptop creates a lot of keys to revoke after compromise.
- By comparison, if only one or two API keys are available in Codespaces, it is far easier to rotate two keys than ten.
- Consider banning local development on laptops once a supported sandboxed development path exists.
- Give developers a "happy path" that uses Codespaces or stripped-down VMs to develop and deploy code.
- Make sure we have a process for installing required development tools in these sandboxed environments, or the control will create major developer frustration.

Cleanroom and Hardened Package Providers

Use a service like Seal Security or Chainguard that can:

- Build packages from source in a cleanroom environment instead of relying directly on npm, PyPI, or Maven artifacts.
- Protect against dependency compromise, such as a child dependency being taken over and having malware inserted into it.
- Backport CVE fixes so developers do not always need to make code changes or absorb breaking upstream upgrades to remediate vulnerabilities.

This does not protect against source code tampering. However, the backporting model is similar to what Red Hat does with RHEL: it is not always directly related to supply chain compromise, but it reduces how vulnerable our software is and can make remediation easier.

Repository Management

Use a software repository manager like Nexus or JFrog Artifactory.

- This can allow us to enforce the use of specific software versions.
- It could be used to implement dependency cooldowns automatically.
- We could force developers to use stable, non-bleeding-edge versions of software and lower the risk of adopting a compromised release.
- Similar to Wiz, JFrog Artifactory gives us visibility into what software versions are being used by developers.

Dependency Cooldowns

Use [Dependency Cooldowns](#) to delay adoption of newly published package versions.

This protects against bleeding-edge versions of software being compromised before maintainers, vendors, and the security community have time to detect the issue.

Containerization and Minimal Images

Encourage developers to containerize their applications.

- Using stripped-down versions of Linux, such as Alpine, can reduce our attack surface.
- The fewer unnecessary dependencies we pull in, the smaller our supply chain attack surface.
- With fewer binaries or tools in the container, an attacker may be unable to complete their attack even if a package is compromised.

Reactive Defenses

- Use the SOC team and Wiz Code to determine whether we are using a compromised version.
- Ask Wiz's Mika agent questions such as: "Are we affected by X campaign? What packages are we using?"
- Look for IOCs, such as contact with specific hostnames or IP addresses, or suspicious files on the filesystem.